A second markdown document just focusing on “performance tuning knobs” (block size, occupancy, divergence) tied back to this hierarchy.

Below is a second markdown document that focuses only on **performance tuning knobs** and ties them back to the GPU hierarchy (SM → partitions → warps → cores) you learned about.

GPU Performance Tuning Knobs for Beginners

This document assumes you already know the basic hierarchy:

- Programming: **Grid → Blocks → Threads → Warps**
- Hardware: **GPU → SMs → SM partitions → Cores**

We now focus on three important knobs:

- Block size
- Occupancy
- Warp divergence

These are not independent; changing one usually affects the others.[1] [2] [3] [4]

1. Block size: how many threads per block?

1.1 What is block size?

- **Block size** = `blockDim.x * blockDim.y * blockDim.z` = number of threads per block.
- Common choices: 64, 128, 256, 512, 1024 (CUDA maximum is typically 1024 threads per block).[2] [4]

A block:

- Runs entirely on a single SM.
- Is internally split into **warps of 32 threads**.[5] [6]

So, for block size `256`:

- 256 / 32 = 8 warps per block.[6] [5]

1.2 Why block size matters for performance

Block size strongly affects:

- **Occupancy**: how many warps/threads can be active on an SM at once. [3] [7] [4]
- **Resource usage per block**: registers, shared memory per block decide how many blocks can fit on an SM simultaneously. [2] [7]
- **Scheduling flexibility**:
  - Very small blocks: the SM may hit a **max blocks per SM** limit and end up with few warps, lowering occupancy. [4] [3]
  - Very large blocks: fewer blocks fit on the SM; if one block stalls, you have fewer other blocks to hide latency. [8] [3]

A practical rule of thumb:

- Start with **128 or 256 threads per block**; these sizes often give good occupancy and scheduling flexibility. [3] [8]
- Then adjust based on profiling data.

1.3 Tying to SM and partitions

- When you choose block size, you're indirectly deciding how many **warps** each SM will have.
- Warps are those groups of 32 threads that are assigned to **SM partitions** and scheduled on cores. [5] [6] [9]
- More warps per SM and per partition give the warp scheduler **more choices** to hide latency. [4] [10] [5]

ASCII sketch:

```
Block size = 256 threads
 └─ 8 warps

SM has resource capacity for, say, 16 blocks
 └─ 16 blocks × 8 warps = 128 warps per SM (theoretical)

These 128 warps are distributed across SM partitions and scheduled on cores.
```

In reality, register and shared memory usage per thread/block usually reduce the actual number of resident blocks and warps. [2] [7] [4]

2. Occupancy: how "full" is each SM?

2.1 What is occupancy?

Occupancy (CUDA definition):

- The ratio:
$$\text{occupancy} = \frac{\text{active warps per SM}}{\text{maximum possible warps per SM}}$$
(Conceptually; tools compute this for you.) [7] [4]

For example:

- If hardware can support 64 warps per SM, and your kernel has 32 active warps per SM, occupancy is 50%. [7]

2.2 Why occupancy matters

- The GPU hides memory and instruction latency by **switching between warps**.
- If a warp stalls (waiting on global memory), the warp scheduler picks another ready warp from the pool. [4] [5] [10]
- Higher occupancy → more warps to choose from → better chance of always having something ready to run → better utilization of cores. [3] [7] [4]

However:

- **Higher occupancy is not the goal; higher performance is.** [8] [7]
- Some kernels are limited by memory bandwidth or instruction dependencies; after a point, increasing occupancy may not help.

2.3 What limits occupancy?

Several factors limit how many threads/blocks/warps can be resident on an SM:

- **Block size** (threads per block). [3] [7]
- **Register usage per thread**:
  - More registers per thread → fewer threads can fit on an SM. [2] [7]
- **Shared memory usage per block**:
  - More shared memory per block → fewer blocks/warps per SM. [7] [4] [2]
- **Hardware limits**:
  - Max warps per SM.
  - Max blocks per SM.
  - Max threads per SM. [3] [7]

CUDA tools like the **occupancy calculator** or Nsight Compute tell you what is limiting occupancy and suggest block sizes. [1] [2] [7]

2.4 Tying to SM partitions and cores

Occupancy is measured per SM but affects how well each **partition and its cores** are used:

- Each SM partition has its own **warp scheduler** and a slice of cores. [11]
- If overall occupancy is low, each partition has few warps, so when one warp stalls, the scheduler may have no ready warp to run. [4] [5] [11]
- If occupancy is high, each partition usually has multiple ready warps, which keeps the cores busy. [5] [10] [4]

You can imagine:

```
Low occupancy:
  Partition 0: 2 warps
  Partition 1: 1 warp
  Partition 2: 1 warp
  Partition 3: 0 warps (idle)

High occupancy:
  Partition 0: 16 warps
  Partition 1: 16 warps
  Partition 2: 16 warps
  Partition 3: 16 warps
```

Under high occupancy, all partitions are more likely to be fully utilized.

3. Warp divergence: when threads disagree

3.1 What is warp divergence?

Warp divergence occurs when threads in the same warp take different control-flow paths (e.g., `if/else`, `switch`, `while`) so that they need to execute different instructions. [12]

Because a warp executes one common instruction at a time:

- If half the threads want to go into `if` and half into `else`, the warp must run **both paths**, masking off inactive threads each time. [13] [12]
- Effective work per cycle is reduced; some lanes (threads) are idle while others execute. [13] [12]

3.2 Why divergence hurts performance

- Divergence reduces the **SIMT efficiency** of the cores:
  - The warp scheduler still issues one instruction per cycle per warp.
  - But fewer lanes (threads) are doing useful work for that instruction. [13] [12]
- In extreme cases, each thread in a warp might follow a different path, causing a big slowdown. [12]

3.3 Typical sources of divergence

- `if` conditions that vary per thread (e.g., `if (data[idx] > threshold) ...`).
- Loops where iteration counts differ per thread.
- Early exits in loops or functions. [13] [12]

3.4 Tying divergence to partitions and cores

From the hardware point of view:

- The **warp scheduler in a partition** issues instructions per warp, not per thread. [5] [11]
- When divergence happens, it must serially execute each path of the branch:
  - Sequence:
    - Run path A with a subset of threads active.
    - Run path B with a different subset active.
- Cores are underutilized during each path because not all lanes are active. [13] [12]

So even if occupancy is high and we have many warps, divergence within each warp directly reduces how efficiently cores in that partition are used.

4. Memory access patterns (bonus knob tied to warps)

You did not explicitly ask for this, but it is as important as the other knobs.

4.1 Coalesced vs uncoalesced access

- **Coalesced access**: Threads in a warp access memory addresses that are contiguous or within a single aligned segment.
- **Uncoalesced access**: Threads in a warp access scattered addresses, causing multiple memory transactions. [14] [12] [4]

Guidelines:

- Make `threadIdx.x` map to consecutive elements in arrays.
- Prefer **structure of arrays (SoA)** over **array of structures (AoS)** for throughput. [4] [14]

4.2 Why it matters for SM and partitions

- Global memory bandwidth is shared across SMs.
- Within an SM, warps of a partition issue memory requests.
- With good coalescing:
  - Each warp's memory instruction turns into minimal memory transactions.
  - Less pressure on memory system, more time for cores to compute. [14] [4]
- With poor coalescing:

    - More transactions per warp.
    - More waiting, so warp schedulers have to rely on high occupancy to hide that latency. [4] [14]

This image of GPU memory hierarchy shows how data moves from global memory through caches to SMs; coalescing helps use that hierarchy efficiently. [4]

5. Practical starting rules for a new engineer

You can think in terms of simple “first try” rules, then refine using profilers.

5.1 Block size rules

- Start with **128 or 256 threads per block.** [3] [8]
- Ensure block size is a multiple of 32 (warp size). [5] [6]
- Avoid very tiny blocks (like 32 threads) unless profiling says otherwise, because they often limit occupancy. [4] [3]

5.2 Occupancy rules

- Use tools (Nsight Compute, occupancy calculator) to see your kernel’s occupancy and its limiting factors. [1] [2] [7]
- If occupancy is low due to:
    - Block size: try increasing threads per block.
    - Registers per thread: consider simplifying code, using `__restrict__`, or rethinking per-thread state. [2] [7]
    - Shared memory: reduce shared memory usage per block or share buffers more carefully. [7] [4] [2]
- Aim for “reasonably high” occupancy, not necessarily the maximum, and confirm with benchmarks. [8] [7]

5.3 Divergence rules

- Try to make branch conditions **uniform within a warp** when possible:
    - Map warps to regions of data that behave similarly.
- Pull complex, highly divergent logic out of the hot path, or refactor the algorithm to reduce branching. [13] [12]
- If divergence is data-dependent and hard to avoid, look at regrouping data so similar cases are processed together by the same warps. [12]

5.4 Memory rules

- Prioritize **coalesced accesses**; design data layout accordingly. [14] [4]
- Use shared memory to cache data reused by many threads in a block. [2] [4]
- Launch enough threads/blocks to keep SMs busy and saturate memory bandwidth. [4]

6. How these knobs connect back to the hierarchy

You can summarize the relationships like this:

- **Block size** decides:
  - How many threads per block → how many warps per block.
  - How many resources each block uses on an SM. [2] [3] [7]
- **Occupancy** captures:
  - How many warps are resident on each SM (and thus per partition's scheduler).
  - How much choice the warp schedulers have to keep the cores busy. [3] [7] [4]
- **Divergence** determines:
  - How efficiently each warp uses its cores once scheduled.
  - Even with high occupancy, bad divergence wastes cycles in the SM partitions. [13] [12]
- **Memory coalescing** affects:
  - How long warps wait on memory operations.
  - How much occupancy and ILP you need to hide that latency. [14] [4]

If you keep this mental model in mind, you can look at a profiler report (occupancy, warp execution efficiency, memory throughput) and directly relate the numbers to:

- SMs and partitions (occupancy/active warps).
- Warps and cores (divergence and instruction throughput).
- Memory system (coalescing and bandwidth).

If you'd like, next I can walk through a concrete kernel (e.g., vector add or 2D stencil) and show, step by step, how I would pick block size, measure occupancy, detect divergence, and improve it.

⁂

1. https://christianjmills.com/posts/cuda-mode-notes/lecture-008/
2. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/
3. https://moderngpu.github.io/performance.html
4. https://www.olcf.ornl.gov/wp-content/uploads/2020/04/04-CUDA-Fundamental-Optimization-Part-2.pdf
5. https://doc.sling.si/en/workshops/programming-gpu-cuda/02-GPU/01-exemodel/

6. https://developer.codeplay.com/products/computecpp/ce/2.6.0/guides/sycl-for-cuda-developers/execution-model.html
7. https://www.youtube.com/watch?v=OIOx3CJP2Es
8. https://news.ycombinator.com/item?id=41808013
9. https://www.abhik.ai/concepts/gpu-computing/shared-multiprocessor
10. https://ajdillhoff.github.io/notes/cuda_architecture/
11. https://modal.com/gpu-glossary/device-hardware/warp-scheduler
12. https://www.sciencedirect.com/topics/computer-science/warp-divergence
13. https://www.youtube.com/watch?v=ypz8hOZ_xLU
14. https://www.youtube.com/watch?v=PRtg7KqVs4A
15. https://www.reddit.com/r/CUDA/comments/1ekin72/which_cuda_block_configuration_is_better_for/